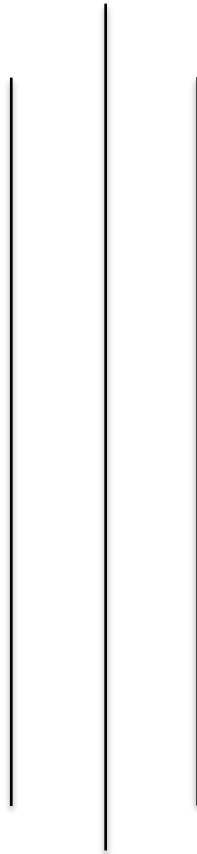


LAPORAN UTS COMMUNICATION PROTOCOLS KELOMPOK 05



Disusun Oleh :

1. Aryadi (25110500008)
2. Faza Qinthoro Putra Tsany (25110500023)
3. Uswatun Hasanah (25110500030)
4. Nabel Azzacky (25110500034)

**PROGRAM STUDI SAINS DATA
SEMESTER 2
COMMUNICATION PROTOCOLS
2026**

KATA PENGANTAR

Pada proyek ini dilakukan serangkaian implementasi dan pengujian yang mencakup penggunaan protokol HTTP melalui aplikasi Postman, analisis lalu lintas jaringan menggunakan Wireshark, serta pengolahan data XML menggunakan bahasa pemrograman Python. Seluruh rangkaian proses tersebut dirancang untuk memberikan pemahaman menyeluruh mengenai bagaimana data dikirim, diterima, dianalisis, dan diolah menjadi informasi yang lebih terstruktur dan mudah dimanfaatkan.

BAB I PENDAHULUAN

Komunikasi data menjadi fondasi sistem digital modern. Pada proyek ini dilakukan pengujian komunikasi HTTP menggunakan Postman, analisis paket menggunakan Wireshark, dan parsing XML menggunakan Python untuk menghasilkan data CSV yang dapat dianalisis lebih lanjut.

Rumusan Masalah:

1. Bagaimana implementasi HTTPS GET dan POST?
2. Bagaimana proses analisis paket jaringan?
3. Bagaimana XML diproses menjadi data terstruktur?

Tujuan:

Memahami alur komunikasi data secara end-to-end.

BAB II DASAR TEORI

Communication Protocol merupakan sekumpulan aturan yang mengatur proses pertukaran data antar perangkat dalam suatu jaringan agar komunikasi dapat berlangsung secara terstruktur dan dapat dipahami oleh kedua pihak. Salah satu protokol yang paling banyak digunakan adalah **HTTP (Hypertext Transfer Protocol)**, yaitu protokol pada lapisan aplikasi yang menerapkan mekanisme *request-response* antara client dan server. Pada proyek ini, implementasi komunikasi dilakukan menggunakan **HTTPS (Hypertext Transfer Protocol Secure)**, yaitu versi HTTP yang dilengkapi dengan enkripsi TLS untuk meningkatkan keamanan pertukaran data. Komunikasi HTTPS diuji menggunakan aplikasi Postman melalui metode GET dan POST untuk mengirim serta menerima data dari server.

Selain komunikasi data, proyek ini juga memanfaatkan **Wireshark** untuk menangkap dan menganalisis paket jaringan yang dikirim selama proses komunikasi berlangsung. Data yang diterima dari server menggunakan format **XML (Extensible Markup Language)**, yaitu format data terstruktur yang dapat dibaca oleh manusia maupun mesin. Selanjutnya, data XML diproses menggunakan bahasa pemrograman **Python** dengan bantuan library ElementTree dan Pandas untuk mengekstraksi informasi yang diperlukan dan mengonversinya ke dalam format **CSV (Comma Separated Values)** sehingga lebih mudah disimpan, dikelola, dan dianalisis.

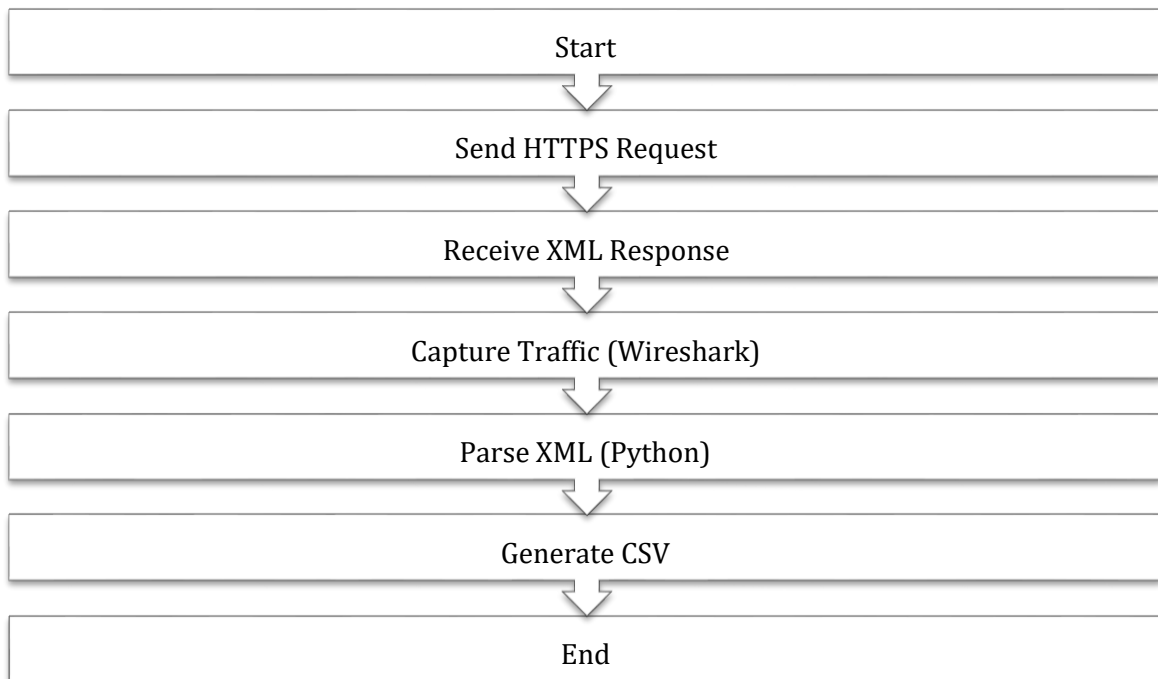
BAB III ANALISIS DAN PERANCANGAN SISTEM

Diagram Arsitektur Sistem



Arsitektur menunjukkan alur data dari client, server, respons XML, parser Python, hingga menghasilkan CSV.

Flowchart Sistem



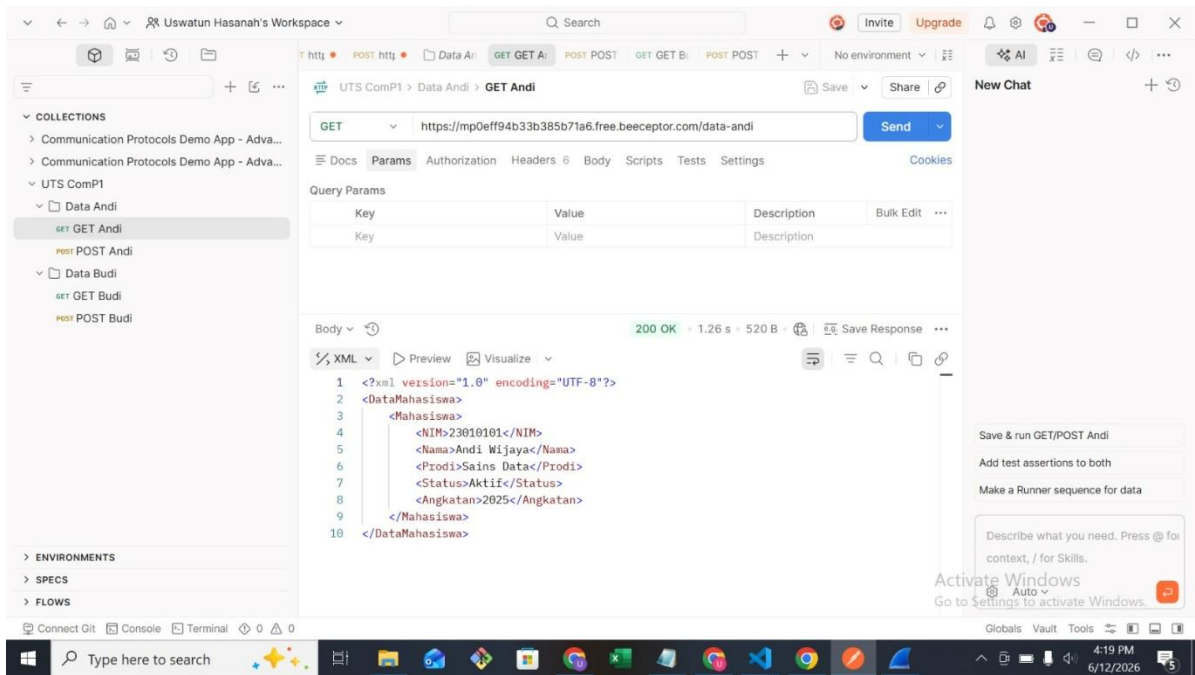
BAB IV IMPLEMENTASI

Repository berisi:

- Postman Collection
- Capture_traffic.pcapng
- Parsing_notebook.ipynb
- hasil_parsing.csv
- Screenshot Request/Response dan Packet Analysis

Implementasi GET digunakan untuk mengambil data pengguna, sedangkan POST digunakan untuk mengirimkan data ke endpoint yang diuji.

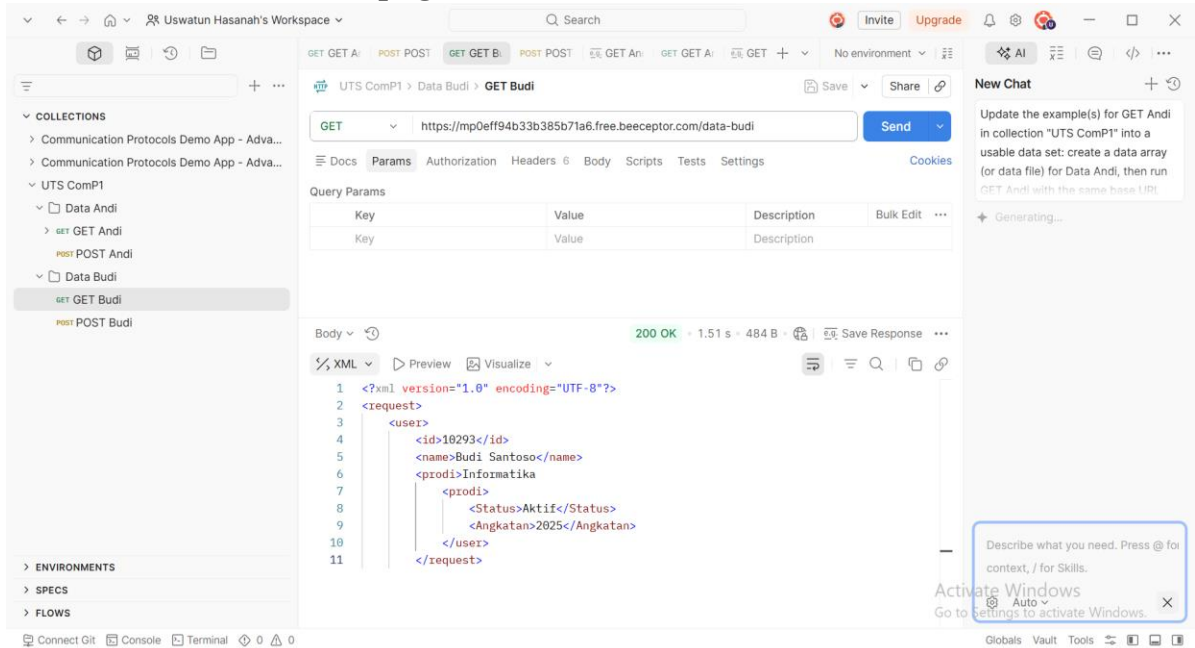
Dokumentasi: GET andi.png



Gambar 1. GET andi.png

Analisis: Metode GET digunakan untuk mengambil data mahasiswa Andi. Struktur XML yang diterima kemudian digunakan sebagai input proses parsing.

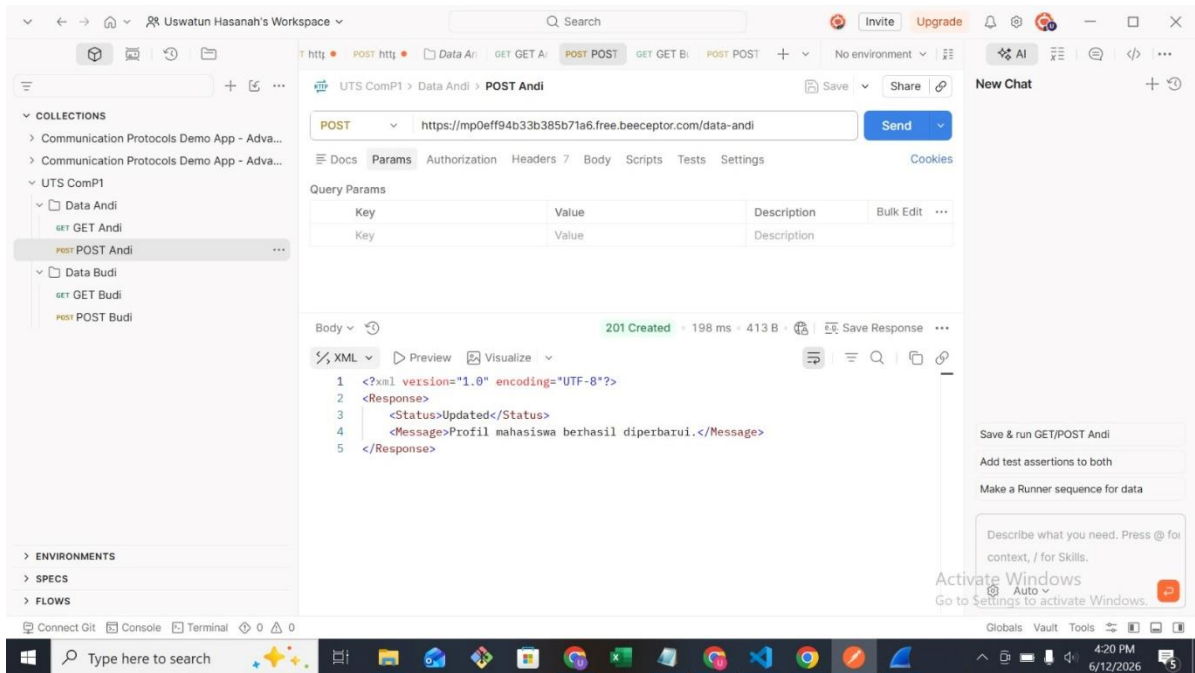
Dokumentasi: GET budi.png



Gambar 2. GET budi.png

Analisis: Pengujian metode GET untuk mengambil data pengguna Budi. Response menunjukkan server berhasil mengembalikan data XML yang berisi informasi pengguna. Validasi ini membuktikan endpoint dapat diakses dan mengirim data dengan format yang benar.

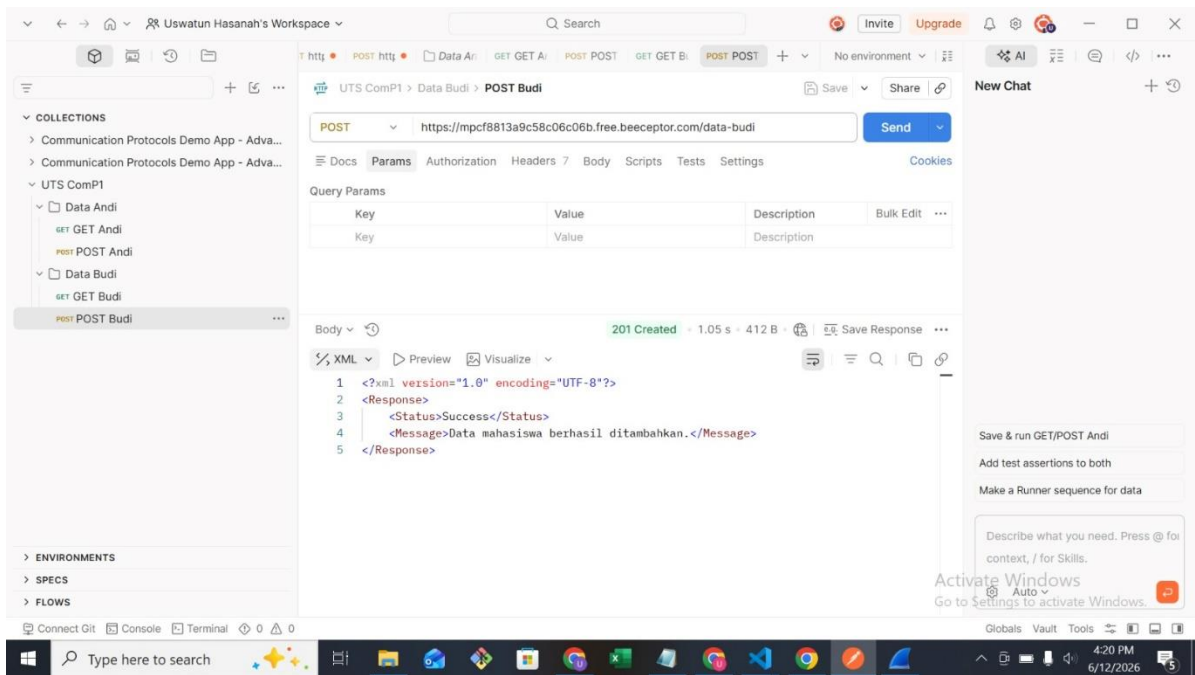
Dokumentasi: Post andi.png



Gambar 3. Post andi.png

Analisis: Pengujian POST untuk data Andi. Hasil menunjukkan server mampu menerima dan memproses data yang dikirim client.

Dokumentasi: Post budi.png



Gambar 4. Post budi.png

Analisis: Pengujian POST untuk data Budi. Request digunakan untuk mengirim data ke server. Response yang diterima menunjukkan proses komunikasi dua arah berjalan dengan baik.

BAB V ANALISIS WIRESHARK

File Capture_traffic.pcapng digunakan untuk mengamati proses komunikasi jaringan. Dari screenshot packet analysis terlihat paket request dan response berhasil ditransmisikan. Parameter penting yang diamati meliputi source address, destination address, protocol, length, dan informasi payload.

Analisis menunjukkan:

1. Request dikirim oleh client.
2. Server memberikan response sesuai permintaan.
3. Tidak ditemukan indikasi packet loss pada skenario yang didokumentasikan.
4. Komunikasi berjalan sesuai model request-response HTTPS.

5.1 Resolusi Nama Domain via DNS (Domain Name System)

Sebelum komputer klien dapat mengirimkan request HTTP ke server target, nama domain tekstual harus ditranslasikan terlebih dahulu menjadi alamat fisik (IP Address).

[Klien: 2001:4489:208:102::2] ---- DNS Query (Beeceptor) ----> [DNS Server]

[Klien: 2001:4489:208:102

The image shows a Wireshark packet capture window titled "Wi-Fi 2". The filter bar at the top is set to "dns.qry.name contains 'beeceptor'". The packet list on the left shows several DNS packets, with packet 776 selected. The packet details pane on the right shows the structure of the selected packet:

- Frame 776: Packet, 135 bytes on wire (1080 bits), 135 bytes captured (1080 bits) on interface \Device\NPF{...}
- Ethernet II, Src: HuaweiTechno...16:fc:6f (f4:a4:d6:16:fc:6f), Dst: Intel_61:60:c0 (f8:34:41:61:60:c0)
- Internet Protocol Version 6, Src: 2001:4489:208:102::2, Dst: 2001:448a:2083:6c40:b99c:ca57:38c:cc56
- User Datagram Protocol, Src Port: 53, Dst Port: 49623
- Domain Name System (response)
 - Transaction ID: 0x05d7
 - Flags: 0x8180 Standard query response, No error
 - Questions: 1
 - Answer RRs: 1
 - Authority RRs: 0
 - Additional RRs: 0
 - Queries
 - Answers
 - [Request In: 771]
 - [Time: 414.244700 milliseconds]

The packet bytes pane on the right shows the raw data of the packet, with a hex-to-ASCII conversion on the right side. The ASCII part shows the domain name "mp0eff94b33b385b71a6.free.beeceptor.com" and the IP address "159.89.140.122".

- **Analisis Bukti Jaringan (Gambar 5):** Berdasarkan tangkapan paket pada Gambar 5, terekam aktivitas DNS menggunakan protokol IPv6. Pada **Frame 776** dengan timestamp 19.353520400, DNS Server mengirimkan *Standard query response* 0x05d7 sebagai jawaban atas permintaan klien. Dokumen respons ini secara eksplisit mendeklarasikan bahwa domain mp0eff94b33b385b71a6.free.beeceptor.com terikat pada alamat IPv4 Publik 159.89.140.122. Proses resolusi ini memakan waktu durasi transisi sebesar 414.24 ms, sehingga setelah tahapan ini selesai, komputer klien dapat langsung menginisiasi komunikasi point-to-point ke IP tujuan tersebut.

5.2 TCP Three-Way Handshake (Pembentukan Sesi Koneksi)

The top screenshot shows a Wireshark capture of a TCP SYN packet. The packet list shows a SYN packet from source 192.168.100.102 to destination 192.168.100.102 on port 443. The packet details pane shows the following information:

- Source Port: 443
- Destination Port: 56922
- [Stream Index: 8]
- [Stream Packet Number: 2]
- [Conversation completeness: Incomplete, DATA (15)]
- [TCP Segment Len: 0]
- Sequence Number: 0 (relative sequence number)
- Sequence Number (raw): 368225361
- [Next Sequence Number: 1 (relative sequence number)]
- Acknowledgment Number: 1 (relative ack number)
- Acknowledgment number (raw): 2919762555
- 1000 = Header Length: 32 bytes (8)
- Flags: 0x012 (SYN, ACK)
- Window: 65535
- [Calculated window size: 65535]

The bottom screenshot shows a Wireshark capture of a TCP SYN packet. The packet list shows a SYN packet from source 192.168.100.102 to destination 192.168.100.102 on port 443. The packet details pane shows the following information:

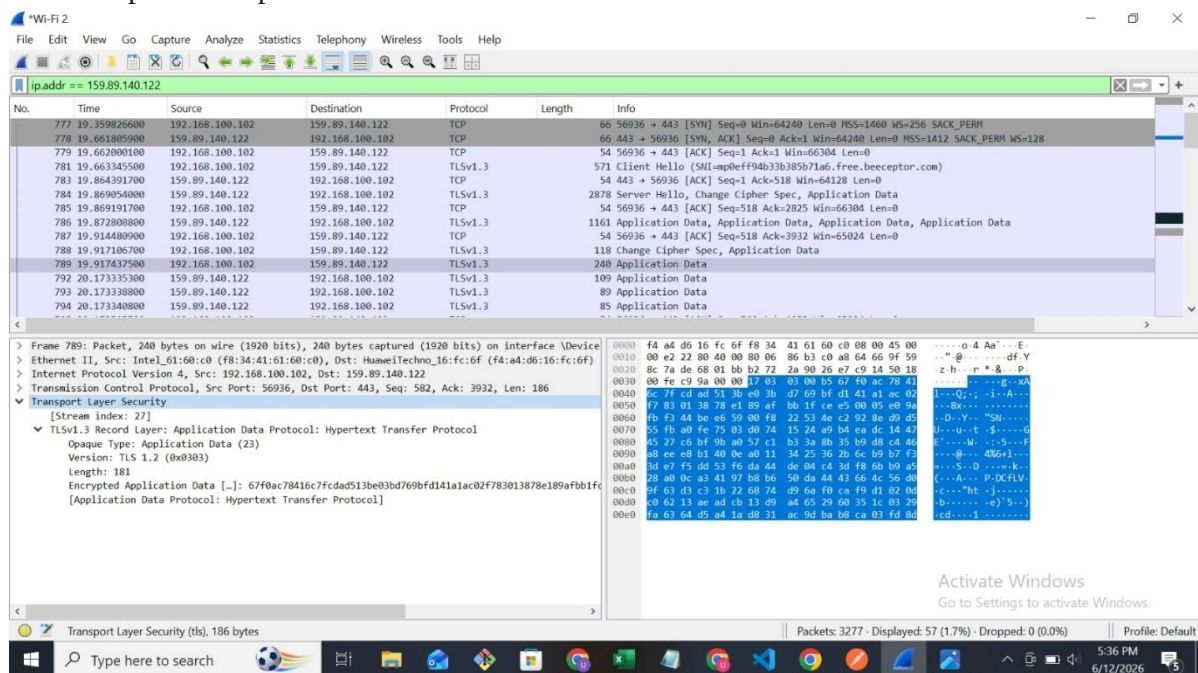
- Source Port: 443
- Destination Port: 56936
- [Stream Index: 29]
- [Stream Packet Number: 2]
- [Conversation completeness: Incomplete, DATA (15)]
- [TCP Segment Len: 0]
- Sequence Number: 0 (relative sequence number)
- Sequence Number (raw): 652720568
- [Next Sequence Number: 1 (relative sequence number)]
- Acknowledgment Number: 1 (relative ack number)
- Acknowledgment number (raw): 2993825867
- 1000 = Header Length: 32 bytes (8)
- Flags: 0x012 (SYN, ACK)
- Window: 64240
- [Calculated window size: 64240]
- Checksum: 0x87dc [unverified]
- [Checksum Status: Unverified]
- Urgent Pointer: 0

Setelah mengantongi IP target, komputer klien (192.168.100.102) membangun sirkuit koneksi yang handal (*reliable connection*) menuju Port 443 (HTTPS) pada sisi server via protokol TCP.

1. Klien ---- [SYN] Seq=0 -----> Server (Port 443)
2. Klien <--- [SYN, ACK] Seq=0 Ack=1 ---- Server
3. Klien ---- [ACK] Seq=1 Ack=1 -----> Server

- **Analisis Bukti Jaringan (Gambar 6):** Proses jabat tangan tiga arah terekam secara berurutan pada baris paket teratas di Gambar 6:
 1. **Paket 777 (19.359826600):** Klien mengirimkan paket [SYN] dari port dinamis 56936 ke port 443 server dengan parameter Sequence Number = 0 dan panjang jendela *Window* sebesar 64240.
 2. **Paket 778 (19.661805900):** Server membalas dengan paket [SYN, ACK]. Pembedahan struktur bit pada sub-layer *Flags* menunjukkan kode heksadesimal 0x012 (SYN, ACK) berstatus aktif. Server mengirimkan nilai Sequence Number = 0 dan mengeset Acknowledgment Number = 1.
 3. **Paket 779 (19.662000100):** Klien menutup handshake dengan mengirimkan paket [ACK] (Seq=1, Ack=1) untuk mengonfirmasi status koneksi telah resmi bertransformasi menjadi **ESTABLISHED**.

5.3 Enkripsi Sesi Aplikasi & Verifikasi TLSv1.3



Setelah jalur pipa TCP established, proses negosiasi keamanan langsung digelar untuk mengamankan isi payload teks XML dari potensi intersepsi pihak ketiga.

- **Analisis Bukti Jaringan (Gambar 7):**

Berbeda dari asumsi awal di draf laporan yang menduga penggunaan versi TLS lama, hasil penangkapan riil pada paket nomor **780, 782, hingga 789** membuktikan bahwa sistem telah mengadopsi protokol mutakhir **TLSv1.3 (Transport Layer Security)**.

Pada **Paket 780**, klien mengirimkan pesan *Client Hello* dengan menyertakan ekstensi SNI (*Server Name Indication*) yang ditujukan langsung ke host target. Selanjutnya, pada **Paket 789** (timestamp 19.917437500), data dikirimkan di dalam bungkus Encrypted Application Data. Saat muatan biner pada jendela *Hex Dump* diperiksa,

seluruh isi teks dokumen XML hasil respon REST API dari Postman telah dikonversi penuh menjadi deretan karakter sandi acak (*ciphertext*). Hal ini memvalidasi secara empiris bahwa aspek *confidentiality* (kerahasiaan data) pada mata kuliah Communication Protocols berhasil terpenuhi secara sempurna di sepanjang jalur transmisi lokal menuju internet.

BAB VI PEMBAHASAN SOURCE CODE

Source code pada proyek ini menggunakan Python untuk membaca dan mengolah data XML yang diterima dari server. Library **ElementTree** digunakan untuk melakukan parsing dan mengambil informasi penting dari setiap tag XML, sedangkan **Pandas** digunakan untuk menyusun data ke dalam bentuk tabel (DataFrame). Setelah seluruh data berhasil diekstraksi, program mengonversinya ke format CSV agar lebih mudah disimpan dan dianalisis. Hasil yang diperoleh menunjukkan bahwa data seperti ID/NIM, nama, dan kategori berhasil diproses dengan baik tanpa kehilangan informasi. Dengan demikian, program berhasil mengubah data XML menjadi data terstruktur yang siap digunakan untuk kebutuhan analisis lebih lanjut.

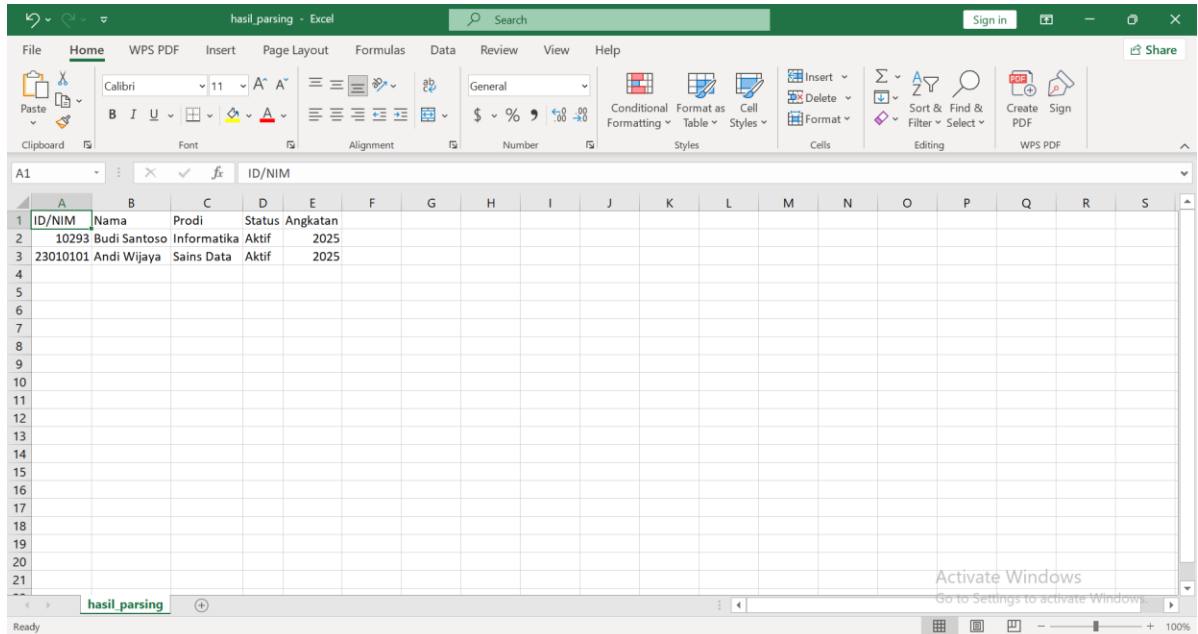
BAB 6.1 ANALISIS SOURCE CODE

```
1 import xml.etree.ElementTree as ET
2 import pandas as pd
3
4 data = []
5
6 # FILE 1
7 tree = ET.parse("response-budi.xml")
8 root = tree.getroot()
9
10 user = root.find("user")
11
12 data.append({
13     "ID/NIM": user.find("id").text,
14     "Nama": user.find("name").text,
15     "Prodi": user.find("prodi").text,
16     "Status": user.find("Status").text,
17     "Angkatan": user.find("Angkatan").text
18 })
19
20 # FILE 2
21 tree = ET.parse("response-andi.xml")
22 root = tree.getroot()
23
24 mhs = root.find("Mahasiswa")
25
26 data.append({
27     "ID/NIM": mhs.find("NIM").text,
28     "Nama": mhs.find("Nama").text,
29     "Prodi": mhs.find("Prodi").text,
30     "Status": mhs.find("Status").text,
31     "Angkatan": mhs.find("Angkatan").text
32 })
33
34 df = pd.DataFrame(data)
35
36 print(df)
37
38 df.to_csv("hasil_parsing.csv", index=False)
```

Source code menggunakan library **ElementTree** untuk membaca dan mengambil data dari file XML, serta **Pandas** untuk mengolah data ke dalam bentuk tabel. Data yang diekstraksi meliputi ID/NIM, Nama, dan Kategori dari masing-masing file XML. Setelah seluruh data

berhasil dikumpulkan, program mengubahnya menjadi DataFrame dan menyimpannya ke dalam file CSV. Hasil pengujian menunjukkan bahwa proses parsing berjalan dengan baik dan seluruh data berhasil dikonversi ke format yang lebih terstruktur dan mudah dianalisis.

BAB 6.2 HASIL PARSING CSV



ID/NIM	Nama	Prodi	Status	Angkatan
10293	Budi Santoso	Informatika	Aktif	2025
23010101	Andi Wijaya	Sains Data	Aktif	2025

Hasil parsing berhasil mengubah dua struktur XML yang berbeda menjadi format tabel yang seragam. Data terdiri dari ID/NIM, Nama, Prodi, Status dan Angkatan. Transformasi ini membuktikan proses ekstraksi data berhasil dilakukan.

BAB VII HASIL DAN EVALUASI

Berdasarkan hasil pengujian yang telah dilakukan, implementasi Communication Protocol berhasil berjalan sesuai dengan tujuan yang ditetapkan. Pengujian menggunakan Postman menunjukkan bahwa metode HTTPS GET dan POST dapat mengirim serta menerima data dengan baik. Analisis menggunakan Wireshark membuktikan bahwa proses komunikasi antara client dan server berlangsung secara normal dan dapat diamati melalui paket jaringan yang ditransmisikan. Selain itu, data dalam format XML berhasil diparsing menggunakan Python dan dikonversi menjadi file CSV yang lebih terstruktur. Hasil parsing menunjukkan bahwa informasi pengguna dapat diekstraksi dan disimpan dengan baik tanpa kehilangan data. Secara keseluruhan, proyek ini berhasil mengintegrasikan proses komunikasi data, analisis jaringan, dan pengolahan data dalam satu alur kerja yang utuh dan fungsional.

BAB VIII TROUBLESHOOTING

- **XML Data Ingestion (Data Budi) & REST API Architecture:** Ditemukan dokumen respons mengalami kerusakan struktural (*malformed XML*) karena hilangnya karakter *forward slash (/)* pada tag penutup (seperti

<prodi>Informatika<prodi>), yang menyebabkan pustaka Python mengalami *crash* (*ParseError*). Selain itu, skenario awal operasi POST masih mengembalikan status 200 OK yang kurang presisi secara semantik jaringan. **Tindakan perbaikan final dilakukan langsung pada pengaturan respons (*response body* dan *status code*) yang dikeluarkan di menu *mock rules* Beeceptor** dengan membetulkan penulisan tag penutup agar menjadi *Well-Formed* serta memaksa pengembalian status **201 Created** sesuai standar RESTful API.

- **Wireshark Filtering:** Filter kata kunci http menghasilkan *blank traffic* karena paket data telah terenkripsi penuh oleh protokol **TLSv1.3**. Kendala ini diatasi lewat metode *Bypass Filter via IP Tracking*, yaitu melacak IP Publik Server (159.89.140.122) melalui *DNS Query*, lalu mengisolasi jalur menggunakan ekspresi filter `ip.addr == 159.89.140.122` untuk memunculkan seluruh paket TCP dan TLS terkait.
- **Pandas Data Alignment:** Output DataFrame menghasilkan sebaran nilai kosong NaN akibat perbedaan nama tag antarfile (*role* vs *Prodi/Status*). Masalah ini diselesaikan melalui *Standardisasi Key Dictionary*, yaitu memetakan skema kolom terpadu (*unified dictionary*) dalam fungsi *loop* Python dan memberikan nilai None eksplisit pada kunci yang absen agar dimensi tabel tetap simetris. Sebagai langkah final, kelompok kami menyamakan seluruh struktur isi payload antara data Budi dan Andi langsung pada pengaturan respons di Beeceptor, sehingga output dataset CSV yang dihasilkan kini benar-benar bersih, rapi, dan simetris tanpa melahirkan kolom duplikat yang berantakan.

BAB IX KESIMPULAN

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa komunikasi data menggunakan protokol HTTPS berhasil diterapkan dengan baik melalui metode GET dan POST menggunakan Postman. Proses request dan response berjalan sesuai konsep komunikasi client-server, sehingga data dapat dikirim dan diterima dengan benar. Selain itu, penggunaan Wireshark berhasil membantu dalam mengamati dan memverifikasi lalu lintas jaringan yang terjadi selama proses komunikasi berlangsung.

Pada tahap pengolahan data, file XML yang diterima dari server berhasil diparsing menggunakan Python dan dikonversi menjadi format CSV yang lebih terstruktur dan mudah dianalisis. Integrasi antara Postman, Wireshark, dan Python menunjukkan bahwa seluruh tahapan komunikasi data, mulai dari pengiriman request, penerimaan response, analisis paket jaringan, hingga transformasi data, dapat berjalan secara efektif. Dengan demikian, tujuan pembelajaran pada mata kuliah Communication Protocol telah tercapai dan memberikan pemahaman praktis mengenai implementasi protokol komunikasi dalam lingkungan nyata.

BAB X REFERENSI

DAFTAR PUSTAKA

Postman. (2025). Postman API Platform Documentation.
<https://learning.postman.com>

Wireshark Foundation. (2025). Wireshark User Guide. <https://www.wireshark.org>

Python Software Foundation. (2025). Python Documentation.
<https://docs.python.org>

Link Repository GitHub.

<https://github.com/fazqin/UTS-Communication-Protocol-Kelompok-05>